CS23334 Fundamentals of Data Science Lab

Experiment 14

Feature Scaling & Preprocessing Techniques in Data Science

Objective: To understand the importance of feature scaling, missing value imputation, one-hot encoding, Standard Scaling (Z-score normalization), and Min-Max Scaling (Range normalization) using Scikit-Learn.

Methodology:

1. **Categorical Imputation & Feature Splitting:** Handle missing categorical values using mode imputation (`df.Country.fillna(mode)`) and split dataset into feature matrix (X) and target vector (y).
2. **Numerical Mean Imputation:** Use `SimpleImputer` to replace missing values in continuous features (Age, Salary) with feature mean values.
3. **One-Hot Encoding:** Convert nominal categorical attributes (Country) into binary dummy arrays using `OneHotEncoder(sparse_output=False)`.
4. **Standard Scaling (Z-score Normalization):** Transform features using `StandardScaler` to achieve zero mean ($\mu = 0$) and unit variance ($\sigma = 1$).
5. **Min-Max Scaling (Range Normalization):** Scale feature values using `MinMaxScaler` onto a bounded range [0, 1].

Code:

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, MinMaxScaler

df = pd.read_csv("pre-process_datasample.csv")
df
df.head()

df.Country.fillna(df.Country.mode()[0], inplace=True)

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

age = SimpleImputer(strategy='mean', missing_values=np.nan)
Salary = SimpleImputer(strategy='mean', missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])
features
```

```
oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
Country

final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
final_set

sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
feat_standard_scaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

OUTPUT: 1. Imputed Numerical Features & One-Hot Encoded Matrix

```
In [1]: df = pd.read_csv("pre-process_datasample.csv")
In [2]: df.Country.fillna(df.Country.mode()[0], inplace=True)
In [3]: features = df.iloc[:, :-1].values
In [4]: age = SimpleImputer(strategy='mean', missing_values=np.nan)
In [5]: Salary = SimpleImputer(strategy='mean', missing_values=np.nan)
In [6]: features[:, [1]] = age.fit_transform(features[:, [1]])
In [7]: features[:, [2]] = Salary.fit_transform(features[:, [2]])
In [8]: features
Out[8]:
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)

In [9]: oh = OneHotEncoder(sparse_output=False)
In [10]: Country = oh.fit_transform(features[:, [0]])
In [11]: final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
In [12]: final_set
Out[12]:
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

OUTPUT: 2. StandardScaler & MinMaxScaler Transformed Feature Sets

```
In [13]: sc = StandardScaler()
In [14]: feat_standard_scaler = sc.fit_transform(final_set)
In [15]: feat_standard_scaler
Out[15]:
array([[ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  7.58874362e-01,  7.49473254e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00, -1.71150388e+00, -1.43817841e+00],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01, -1.27555478e+00, -8.91265492e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00, -1.13023841e-01, -2.53200424e-01],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,  1.77608893e-01,  6.63219199e-16],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01, -5.48972942e-01, -5.26656882e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,  0.00000000e+00, -1.07356980e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  1.34013983e+00,  1.38753832e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,  1.63077256e+00,  1.75214693e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01, -2.58340208e-01,  2.93712492e-01]])

In [16]: mms = MinMaxScaler(feature_range=(0, 1))
In [17]: feat_minmax_scaler = mms.fit_transform(final_set)
In [18]: feat_minmax_scaler
Out[18]:
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Result:

Feature scaling was successfully applied using both Standard Scaler (Z-score) and Min-Max Scaler.
Categorical attributes were one-hot encoded, and numerical missing values were imputed with column means.
The scaled features showed zero mean and unit variance, confirming correct StandardScaler application.